BEARS Rocket Solver (Semi-Final)

A Sufficiently General Rocket Calculator

October 20, 2026

OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Outline

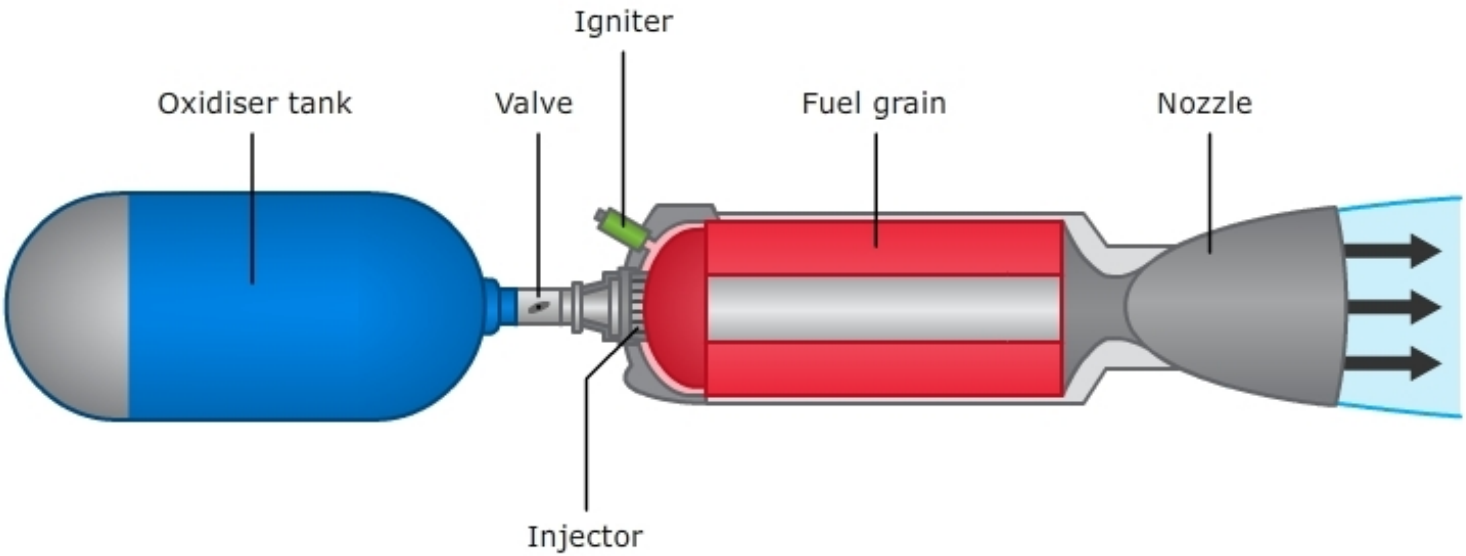
OpenMDAO Framework
- Components
- Groups
- Problems

Rocket Assembly
- Hybrid Engine Nonlinearity

Program Layout
- Modules
- Checks Module

Next steps

Hybrid rocket engine structure



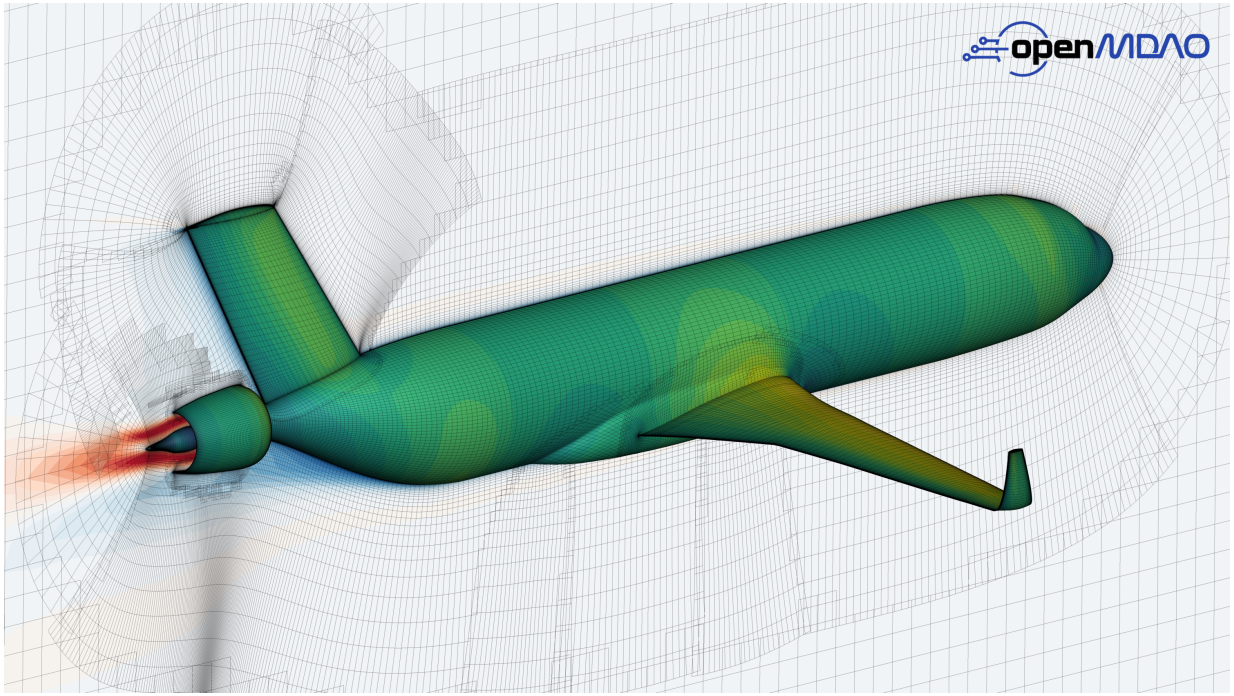
© Copyright. 2011. University of Waikato. All Rights Reserved.

Calculations program requirements

- *Readable:* more than one person needs to be able to understand it
- *Extensible:* working groups should be able to extend their parts without impacting the rest of the assembly
- *General:* not committing to a particular design

OpenMDAO framework

Solution: **OpenMDAO!**

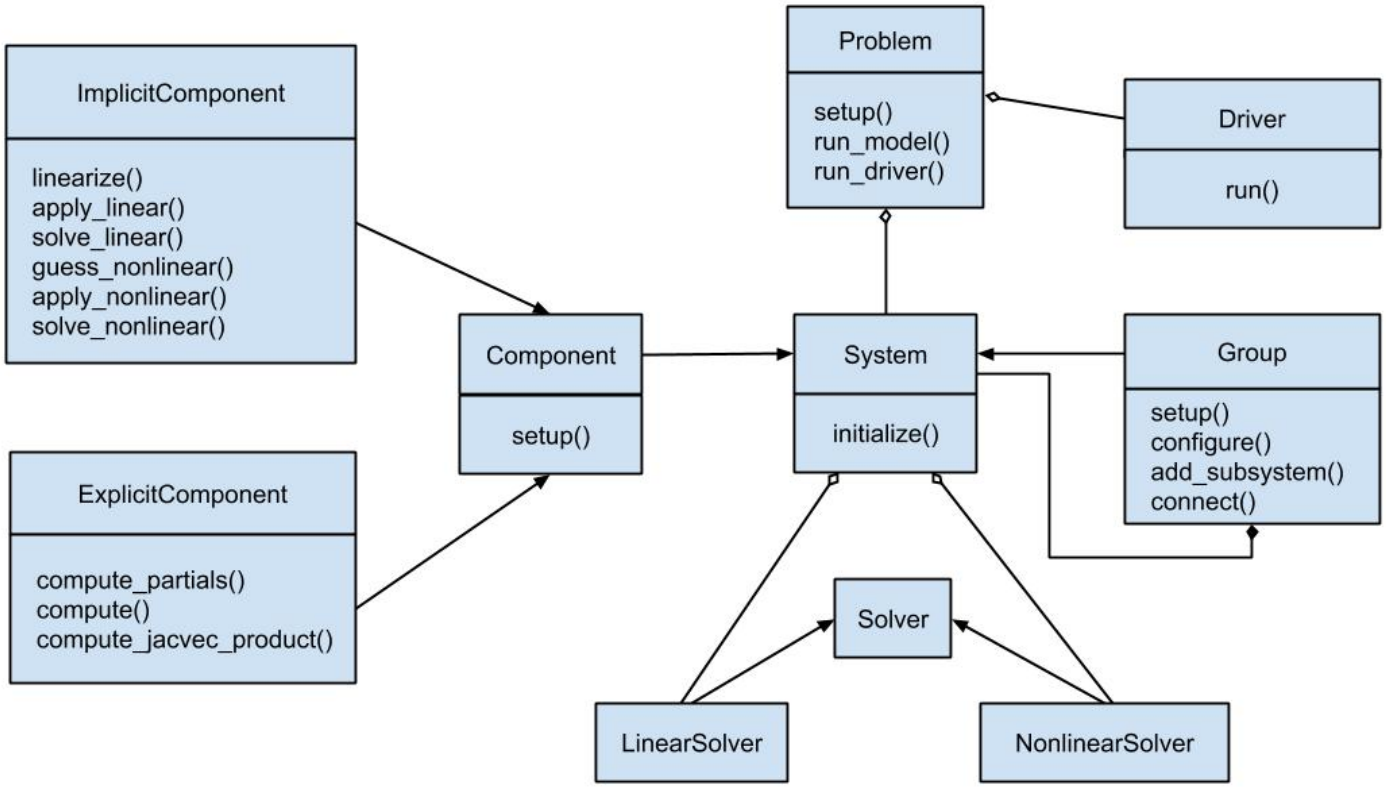


(Multi-Disciplinary Analysis and Optimization)

- Python OOP
- Industry standard

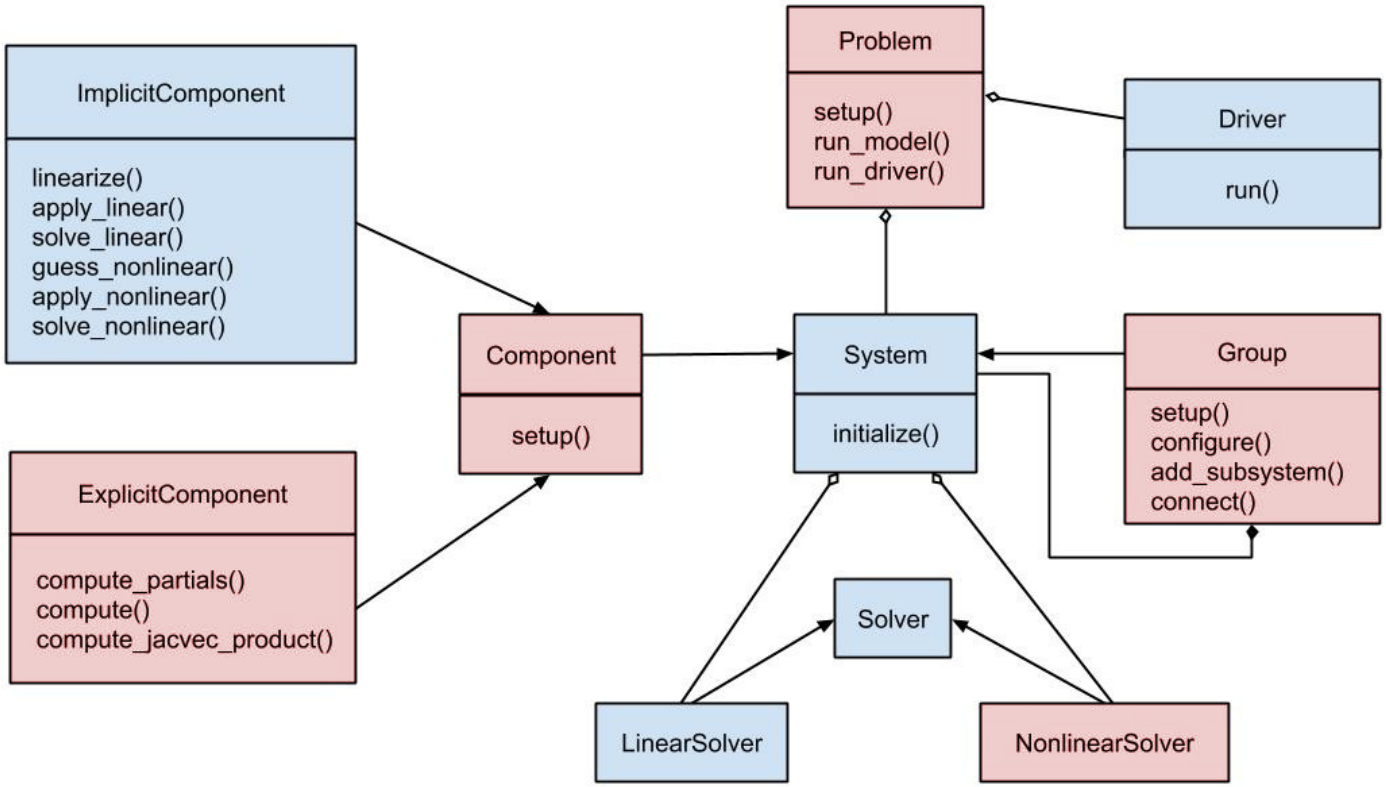
OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO class structure



OpenMDAO class structure

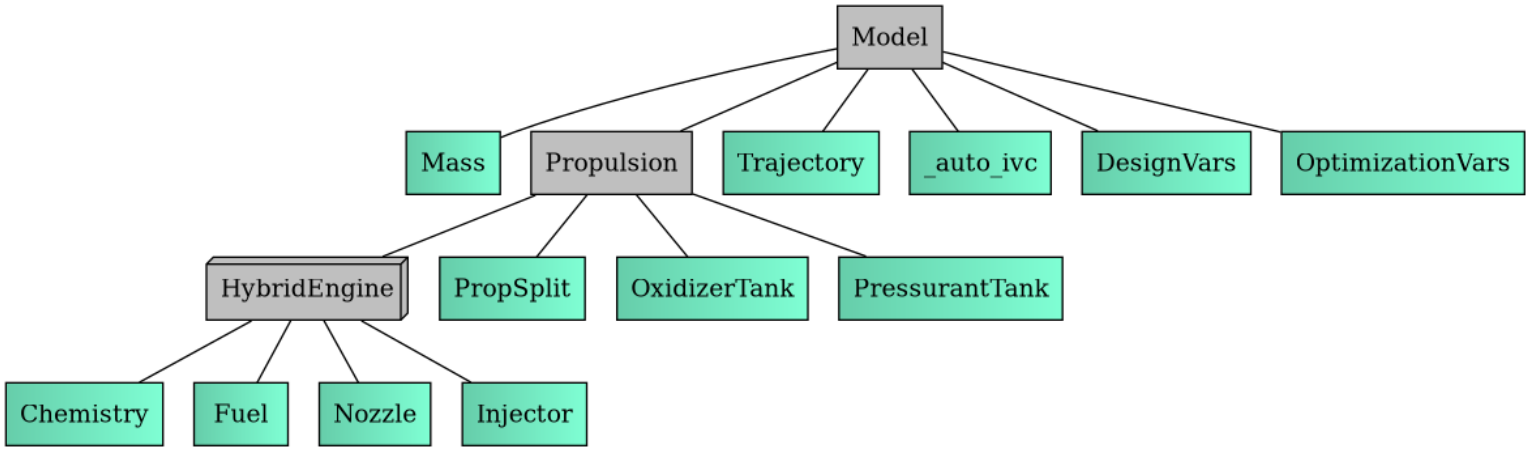
(These are the parts we care about)



OpenMDAO class structure

This assembles into a tree-like structure with

- `Problem`'s internal Model class at root,
- `Group` intances as branching nodes, and
- `Component` instances as leaves:



OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO class structure

Readability: What does the code for this look like?

OpenMDAO class structure

Readability: What does the code for this look like?

In the following: *OpenMDAO by example*

OpenMDAO Component class

```
# Components are the individual assembly units
# that are a responsibility of a single working group

class FooComponent(ExplicitComponent):

    def initialize(self):
        self.options.declare("foo", default=10.0, types=float)

    def setup(self):
        self.add_input("bar",  val=1.0, units="m")
        self.add_input("baz",  val=1.0, units="m")

        self.add_output("xyz", val=2.0, units="m")
        self.add_output("ijk", val=2.0, units="m")

    def setup_partials(self):
        self.declare_partials("*", "*", method="cs")

    def compute(self, inputs, outputs):
        bar = inputs["bar"]

        foo = self.options["foo"]

        baz = calc_baz(foo, bar)

        outputs["baz"] = baz
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO Component class

```
def initialize(self):
    # This is where you put immutable and
    # technical arguments of the class

    self.options.declare("foo", default=10.0,
                         types=float)
```

OpenMDAO Component class

```
def setup(self):
    # This is where you define the way your part
    # interfaces to the rest of the assembly

    # Inputs: data that you need from the assembly
    self.add_input("bar",  val=1.0, units="m")
    self.add_input("baz",  val=1.0, units="m")

    # Outputs: data you provide to the assembly
    self.add_output("xyz", val=2.0, units="m")
    self.add_output("ijk", val=2.0, units="m")
```

OpenMDAO Component class

```
def compute(self, inputs, outputs):
    # This is where your inputs
    # get mapped to your outputs

    # Collect inputs and options, ...
    bar = inputs["bar"]
    baz = inputs["baz"]
    foo = self.options["foo"]

    # ... perform calculations on them, ...
    xyz, ijk = calc_foo(foo, bar, baz)

    # ... and assign the results to the outputs
    outputs["xyz"] = xyz
    outputs["ijk"] = ijk
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO Group class

```
# Groups assemble and connect components together,
# and have to deal with the flow of data between components

class QuuxGroup(Group):

    def initialize(self):
        self.options.declare("foo", default=10.0, types=float)

    def setup(self):
        self.add_input("bar",  val=1.0, units="m")

        self.add_subsystem(
            "Foo",
            FooComponent(foo),
            promotes_inputs  = [("bar", "foo_bar")],
            promotes_outputs = [("xyz", "foo_xyz")],
        )

        self.add_subsystem(
            "Faz",
            FazComponent(foo),
            promotes_inputs  = [("bar", "faz_bar")],
            promotes_outputs = [("xyz", "faz_xyz")],
        )

        self.connect("Foo.ijk", "Faz.baz")
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO Group class

```
# Groups can promote inputs and outputs of their
# components, in which case the promoted
# input (output) becomes the input (ouptut)
# of the whole group viewed as a component

self.add_subsystem(
    "Foo",
    FooComponent(foo),
    promotes_inputs  = [("bar", "foo_bar")],
    promotes_outputs = [("xyz", "foo_xyz")],
    # . . . . . ^~~~~~~~~~~~~~~~~^
    # This tuple syntax instructs OpenMDAO
    # to rename inputs and outputs
    # to avoid naming clashes
)
```

OpenMDAO Group class

```
# Groups can connect inputs and outputs
# of their components

self.connect("Foo.ijk", "Faz.baz")

# In the ideal case, inputs and outputs
# are mapped cleanly without circular dependencies
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO Group class

```
# These connections can sometimes create
# circular relations, which have to be
# handled with nonlinear solvers

self.nonlinear_solver
  = om.NewtonSolver(solve_subsystems=False)

self.nonlinear_solver.options["maxiter"] = 30
self.nonlinear_solver.options["rtol"]    = 1e-6
self.nonlinear_solver.options["iprint"]  = 0

self.nonlinear_solver.linesearch
  = om.ArmijoGoldsteinLS()

self.linear_solver = om.DirectSolver()

# This is an example of a very basic Newton's
# method solver from our Propulsion group.
# It is simple and efficient
# under the right conditions
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

OpenMDAO Problem class

A `Problem` is a special kind of top-level `Group`

```
class HogeProblem(Problem):
    def __init__(self, foo, **kwargs):
        super().__init__(**kwargs)

        # A Problem models a top-level
        # Group or Component
        self.model = QuuxGroup(foo = foo)

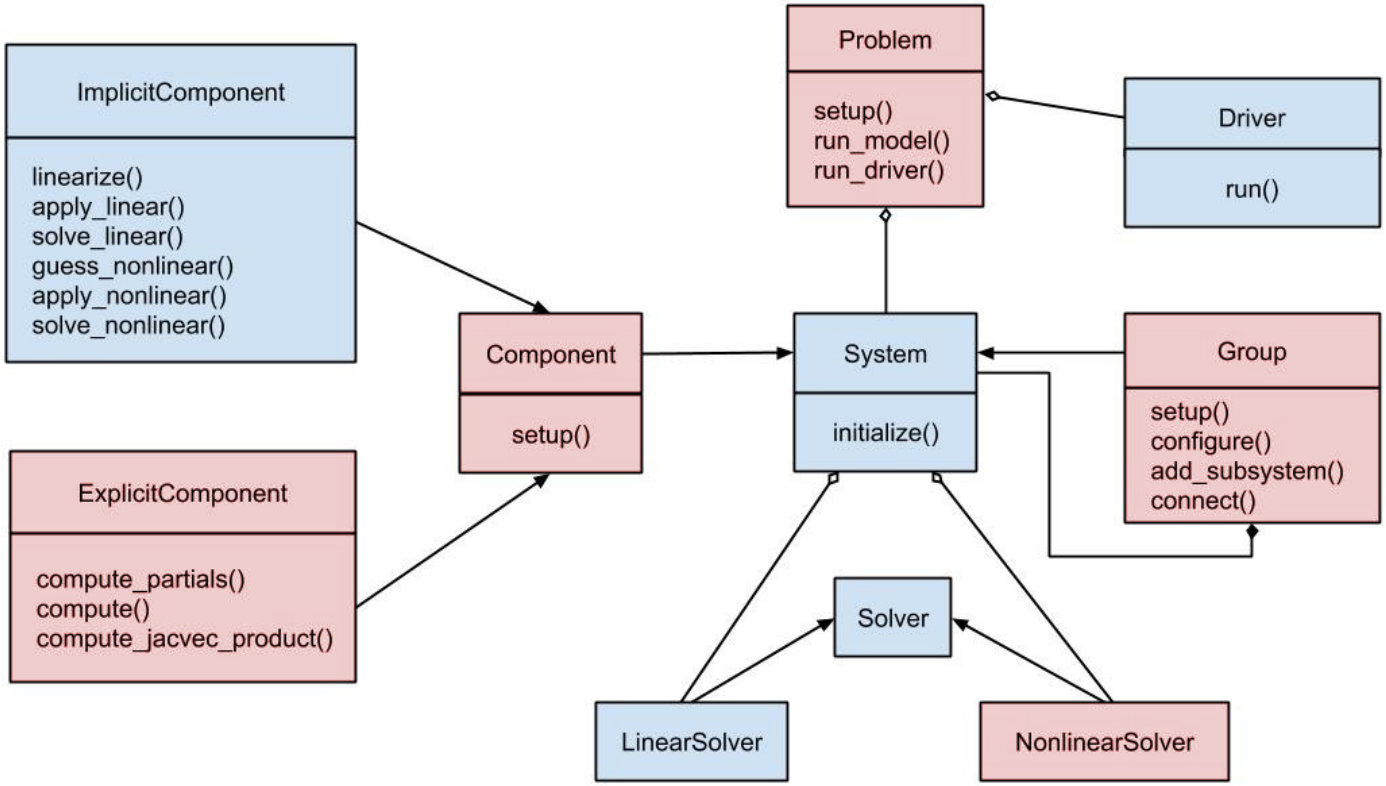
        # It also contains a driver for any
        # non-linear optimization
        # happening internally
        self.driver = om.ScipyOptimizeDriver()
        self.driver.options["optimizer"] = "SLSQP"

# Rest of the code can retrieve optimized values
# after running the optimization driver
hp = HogeProblem(foo)
hp.run_driver()
xyz_opt = hp.get_val("QuuxGroup.foo_xyz")[0]
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

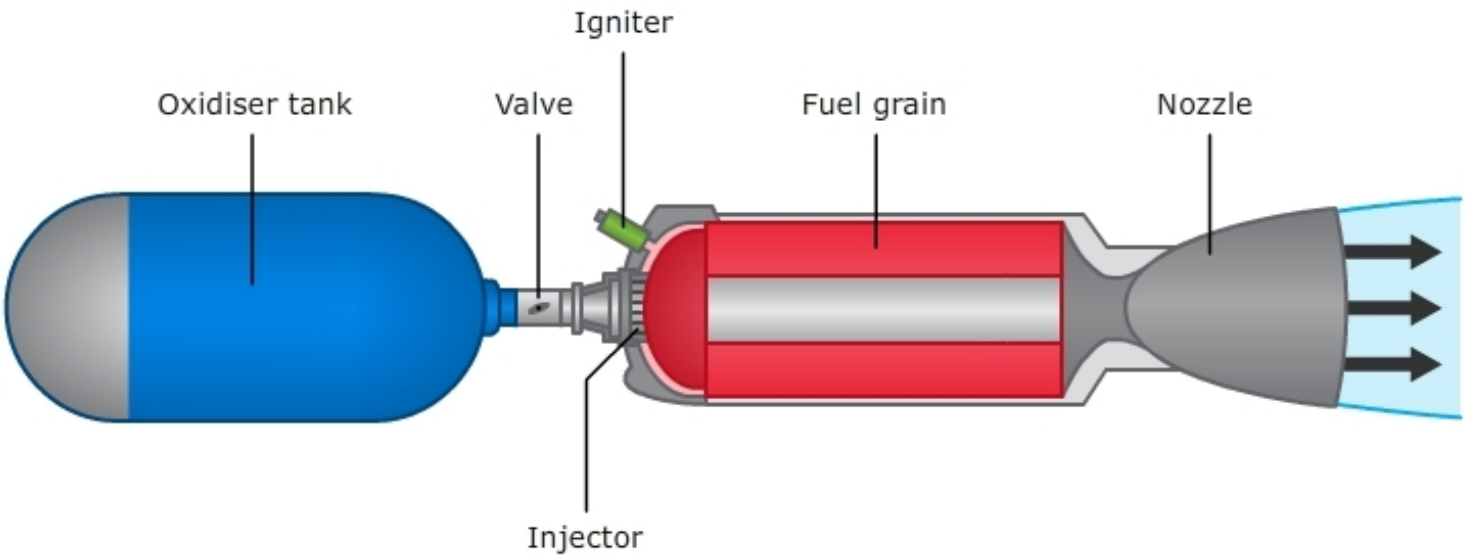
Rocket assembly, class structure

Reminder: OpenMDAO class structure



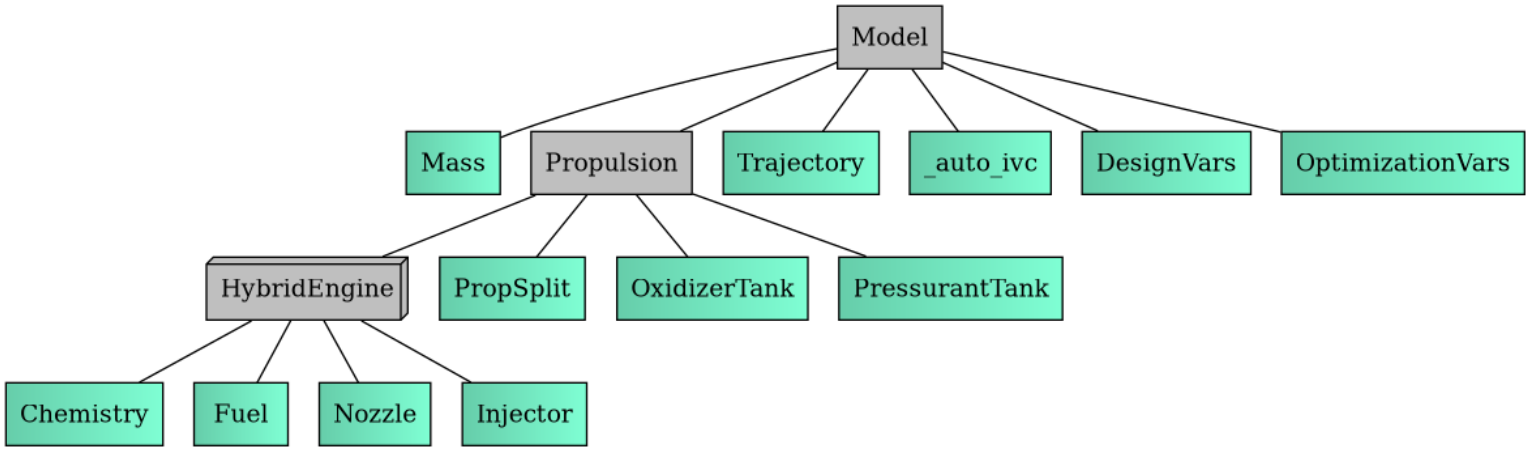
Hybrid rocket engine structure

Reminder: hybrid engine rocket structure



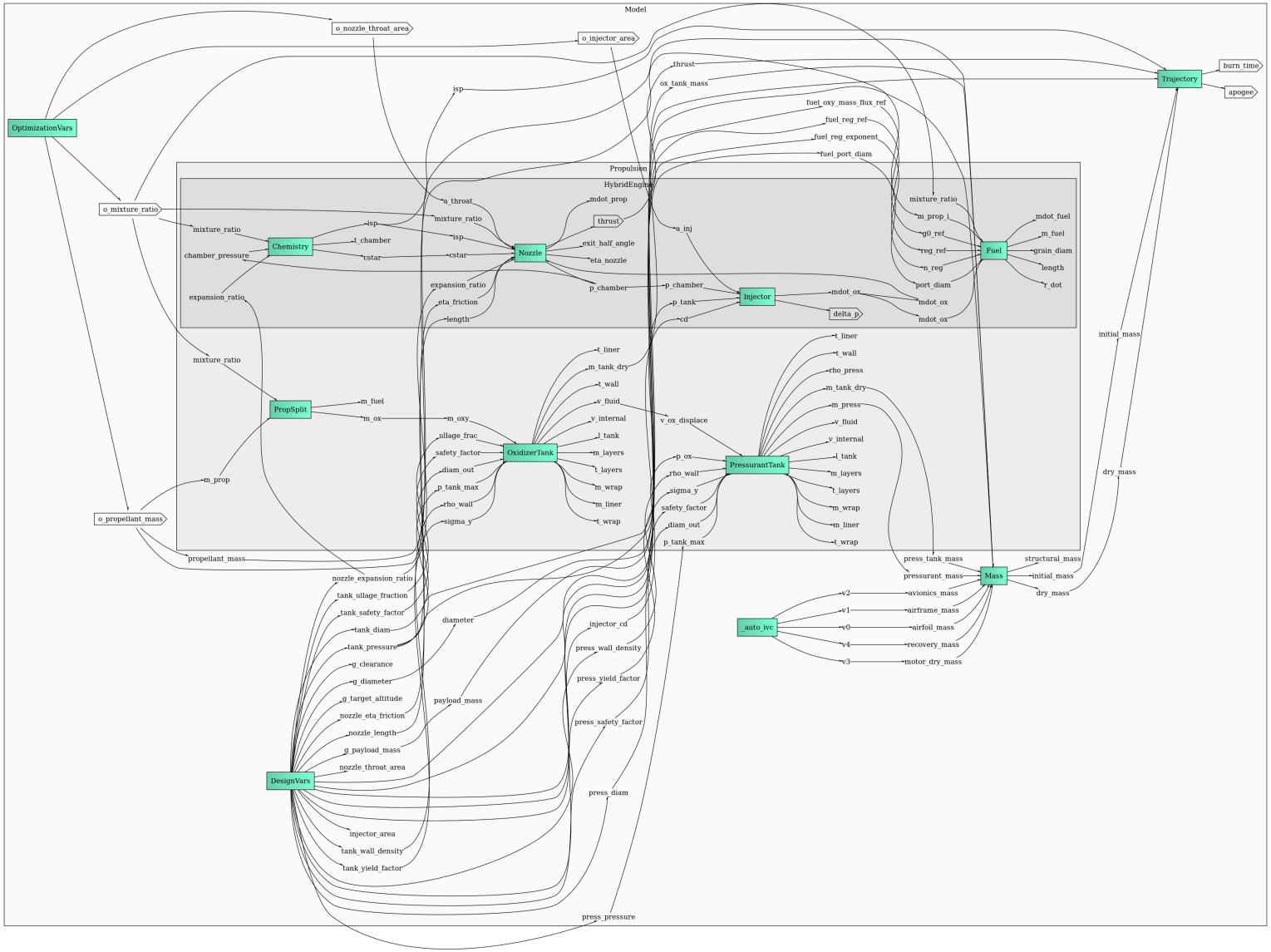
© Copyright. 2011. University of Waikato. All Rights Reserved.

Rocket assembly, class structure



OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Rocket assembly, data flow diagram



Hybrid engine nonlinearity

The components of the `HybridEngine` group form a closed loop:

Injector Incompressible orifice plate flow:

$$\dot{m}_{ox} = C_d A_{inj} \sqrt{2\rho_{ox}(P_{tank} - P_c)}$$

(Source: https://en.wikipedia.org/wiki/Orifice_plate#Incompressible_flow)

Chemistry Gibbs free energy equilibrium (solved internally by the CEA library):

$$c^* = f_{CEA}(P_c, \mathrm{MR})$$

Nozzle Choked flow:

$$P_c = \frac{\dot{m}_{prop} \cdot c^*}{A_t} = \frac{\dot{m}_{ox}\left(1 - \frac{1}{\mathrm{MR}}\right) c^*}{A_t}$$

(Source: https://en.wikipedia.org/wiki/Choked_flow)

Hybrid engine master equation

Substitute the injector and chemistry equations into the nozzle equation:

$$P_c = \left[C_d \frac{A_{inj}}{A_t} \left(1 + \frac{1}{\mathtt{MR}} \right) \right] \cdot \sqrt{2\rho_{ox}(P_{tank} - P_c)} \cdot c^*(P_c, \mathtt{MR})$$

Hybrid engine solver

To solve this, use the residual function $R(P_c) = 0$:

$$R(P_c) = P_c - \left[\frac{A_{inj}}{A_t} C_d \left(1 + \frac{1}{\mathrm{MR}} \right) \cdot \sqrt{2\rho_{ox}(P_{tank} - P_c)} \cdot c^*(P_c, \mathrm{MR}) \right] = 0$$

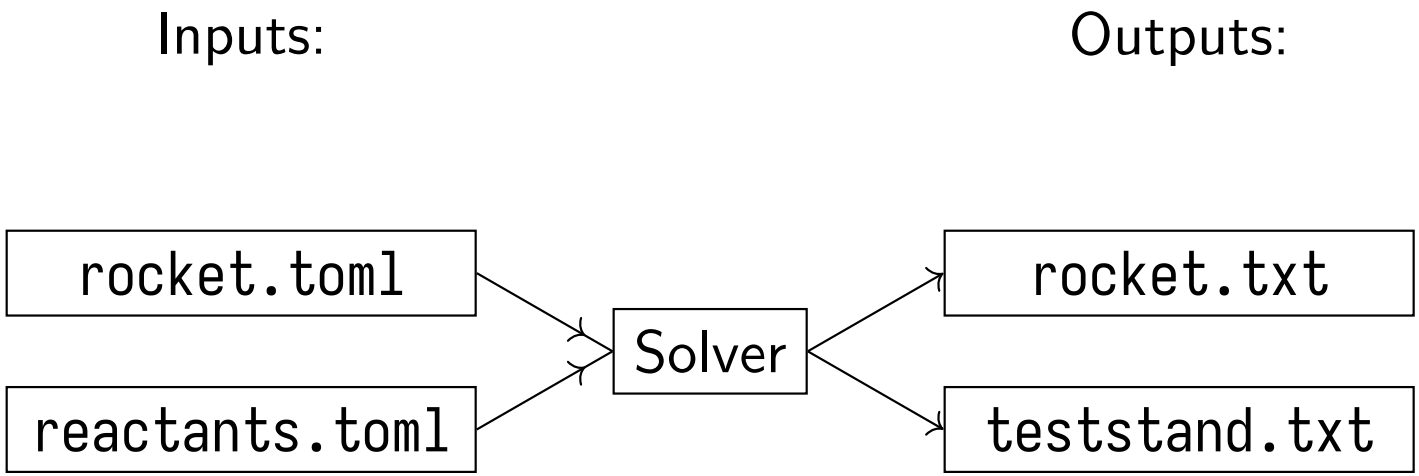
OpenMDAO's `NewtonSolver` solves $R(P_c) = 0$ using an initial guess and tangent lines:

$$P_c^{(k+1)} = P_c^{(k)} - \frac{R(P_c^{(k)})}{\mathrm{d}R/\mathrm{d}P_c}$$

This usually converges in $\mathcal{O}(10)$ steps

OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Rocket assembly, program structure



Program inputs

The input files are configs in human-readable format:

```
[global]
payload_mass_kg          = 1.0
target_altitude_m        = 3000.0
diameter_m               = 0.12
clearance_m              = 0.0

[oxidizer_tank]
pressure_bar             = 70.0
diam_m                   = 0.12
safety_factor            = 2.0
ullage_fraction          = 0.1
yield_factor_pa          = 2.76e8
wall_density_kgm3        = 2700.0

[pressurizer_tank]
pressure_bar             = 200.0
diam_m                   = 0.12
safety_factor            = 2.0
yield_factor_pa          = 2.76e8
wall_density_kgm3        = 2700.0

[injector]
area_m2                  = 4.5e-5
cd                       = 0.7
.
.
.
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Program outputs

The output files are text printouts of OpenMDAO structure:

```
75 Explicit Output(s) in 'model'

varname                  val                  units      prom_name
-----------------------  -------------------  ---------  --------------------------------
DesignVars
  g_payload_mass         [1.]                 kg         DesignVars.g_payload_mass
  g_target_altitude      [3000.]              m          DesignVars.g_target_altitude
  tank_pressure          [70.]                bar        DesignVars.tank_pressure
  tank_diam              [0.12]               m          DesignVars.tank_diam
  tank_safety_factor     [2.]                 None       DesignVars.tank_safety_factor
  .
  .
  .

Propulsion
  PropSplit
    m_ox                 [1.54084888]         kg         Propulsion.PropSplit.m_ox
    m_fuel               [0.16938875]         kg         Propulsion.PropSplit.m_fuel
  OxidizerTank
    t_wall               [0.00147395]         m          Propulsion.tank_wall_thickness
    t_liner              [0.001]              m          Propulsion.tank_liner_thickness
    t_wrap               [0.00047395]         m          Propulsion.tank_wrap_thickness
    m_liner              [0.17283683]         kg         Propulsion.OxidizerTank.m_liner
    m_wrap               [0.04801895]         kg         Propulsion.OxidizerTank.m_wrap
    .
    .
    .
```

Rocket assembly, module structure

```
solver.py
inputs/
└── ...
outputs/
└── ...
modules/
├── BEARS_Atmo
├── BEARS_Chem
├── BEARS_Materials
├── BEARS_Rocket
└── Config
```

The program is laid out in a number of modules according to their purpose

Most `compute()` functions carry out their computations in a few lines, but some computations are too long or require external libraries and have been offloaded to separate modules

Rocket assembly, modules

BEARS_Rocket Core logic with the OpenMDAO model

Rocket assembly, modules

BEARS_Rocket Core logic with the OpenMDAO model

BEARS_Atmo Atmospheric model, offloaded to `ISACALC`

Rocket assembly, modules

BEARS_Rocket Core logic with the OpenMDAO model

BEARS_Atmo Atmospheric model, offloaded to `ISACALC`

BEARS_Chem Reaction chemistry, offloaded to `RocketCEA`

Rocket assembly, modules

BEARS_Rocket Core logic with the OpenMDAO model

BEARS_Atmo Atmospheric model, offloaded to `ISACALC`

BEARS_Chem Reaction chemistry, offloaded to `RocketCEA`

BEARS_Materials Tank sizing and material calculations

OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Rocket assembly, modules

BEARS_Rocket Core logic with the OpenMDAO model

BEARS_Atmo Atmospheric model, offloaded to `ISACALC`

BEARS_Chem Reaction chemistry, offloaded to `RocketCEA`

BEARS_Materials Tank sizing and material calculations

Config Config loader technical module

Rocket assembly, modules

`BEARS_Rocket` Core logic with the OpenMDAO model

`BEARS_Atmo` Atmospheric model, offloaded to `ISACALC`

`BEARS_Chem` Reaction chemistry, offloaded to `RocketCEA`

`BEARS_Materials` Tank sizing and material calculations

`Config` Config loader technical module

(The `BEARS_*` prefix is omitted for `Config` since it contains no meaningful physics)

TODO: `BEARS_Materials` possibly inconsistent/counterintuitive, it contains tank sizing logic and not just material properties

Rocket assembly, core module

```
modules/BEARS_Rocket/
├── checks.py
├── comp_chem.py
├── comp_fuel.py
├── comp_injector.py
├── comp_mass.py
├── comp_nozzle.py
├── comp_split.py
├── comp_tank_oxy.py
├── comp_tank_press.py
├── comp_tank.py
├── comp_traj.py
├── group_engine.py
├── group_prop.py
├── group_rocket.py
├── group_teststand.py
├── prob_rocket.py
└── prob_teststand.py
```

The core module is laid out in one-class-per-file structure following the OpenMDAO class layout

Rocket assembly, core module

```
modules/BEARS_Rocket/
├── checks.py
├── comp_chem.py
├── comp_fuel.py
├── comp_injector.py
├── comp_mass.py
├── comp_nozzle.py
├── comp_split.py
├── comp_tank_oxy.py
├── comp_tank_press.py
├── comp_tank.py
├── comp_traj.py
├── group_engine.py
├── group_prop.py
├── group_rocket.py
├── group_teststand.py
├── prob_rocket.py
└── prob_teststand.py
```

The core module is laid out in one-class-per-file structure following the OpenMDAO class layout

TODO: trajectory component is somewhat large and can be offloaded to a dedicated module

Checks module

checks.py:

Keeping with the goal of extensibility: dedicated module for team members to implement post-run sanity checks

```
class CheckResult(NamedTuple):
    severity : Literal["error", "warning"]
    message  : str

CheckFn
  = Callable[[Problem], tuple[CheckResult, ...]]
```

Check functions

```python
# Example: launch TWR sanity check

def check_liftoff_twr(prob: Problem) -> tuple[CheckResult, ...]:
    issues: list[CheckResult] = []

    thr = prob.get_val("Propulsion.prop_thrust", units="N")[0]
    mi  = prob.get_val("Mass.initial_mass",      units="kg")[0]

    twr = thr / (mi * g)

    if twr < 1.0:
        issues.append(CheckResult(
            "error",
            f"Liftoff TWR below 1.0 ({twr:.3f})"
        ))
    elif twr < 1.3:
        issues.append(CheckResult(
            "warning",
            f"Liftoff TWR is low ({twr:.3f})"
        ))

    return tuple(issues)
```


OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Next steps

We consider the current state of the program mostly production-ready with few exceptions

- **Get subgroup members to start editing system components**
- Drag calculations are long and can be offloaded to a dedicated module
- Materials module rewrite with Rishabh's new code
- Lots of small details need fixing in individual modules
- Better naming?

OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Calculations people



Andrii



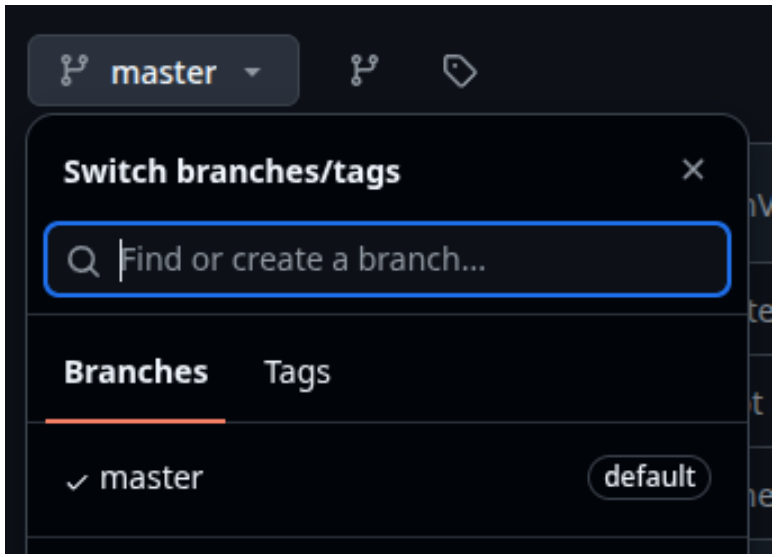
Noa



Sophia

Next steps

Once the current calculations program is fleshed out, we will proceed with merging the solver with BEARS' flight sim program `rocketpal` on a new branch, while master with the current structure remains for support



OpenMDAO Framework
Components
Groups
Problems
Rocket Assembly
Hybrid Engine Nonlinearity
Program Layout
Modules
Checks Module
Next steps

Integration plan

